THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 21

Structured Streaming — Triggers, Checkpoints, and Cost Control

SECTION 01

What Is Structured Streaming

Structured Streaming is Apache Spark's engine for processing continuously arriving data. It uses the same DataFrame and Dataset APIs as batch processing, so the same code you'd write for a static table can, with small changes, run against a live, incrementally updating stream.

CORE CONCEPT

Apache Spark Structured Streaming processes data incrementally through micro-batches. Rather than treating a stream as fundamentally different from a batch job, Structured Streaming treats a stream as a table that keeps growing, and repeatedly runs the same query against whatever new data has arrived.

SECTION 02

Triggers — Controlling When Micro-Batches Run

Trigger intervals control how frequently Structured Streaming checks for new data. Choosing the right trigger mode is a direct lever for balancing latency against cost.

Trigger Mode	Syntax Example	Best For
Unspecified (Default)	N/A	General-purpose streaming with 3-5 second latency; checks for new data continuously
Processing Time	.trigger(processingTime='10 seconds')	Balancing cost and performance by checking on a fixed interval
Available Now	.trigger(availableNow=True)	Scheduled incremental batch processing; processes all currently available data, then stops

THE COST TRAP

By default, if you don't set a trigger mode, Structured Streaming sets the trigger mode to processingTime with an interval of 0, which checks for new data every few milliseconds. This can generate a high volume of cloud storage API calls per day and result in unexpected charges from your cloud provider. Databricks recommends always configuring a trigger mode appropriate for your latency and cost requirements.

SECTION 03

AvailableNow — The Modern Replacement for Trigger.Once

For workloads that don't need continuous processing, such as an hourly or daily incremental load, Trigger.AvailableNow is the current recommended approach.

IMPORTANT NUANCE

In Databricks Runtime 11.3 LTS and later, Trigger.Once is deprecated. Use Trigger.AvailableNow for all incremental batch processing workloads instead. The AvailableNow trigger consumes all available records as an incremental batch, with the ability to configure batch size using options such as maxBytesPerTrigger, processing the backlog across multiple internal batches rather than one single, potentially enormous batch.

```
# Recommended: scheduled incremental batch processing
(spark.readStream
    .format("delta")
    .load("/path/to/bronze")
    .writeStream
    .format("delta")
    .option("checkpointLocation", "/path/to/checkpoint")
    .trigger(availableNow=True)
    .start("/path/to/silver"))
```

SECTION 04

Checkpoints — How Streaming Recovers From Failure

Checkpoints and write-ahead logs work together to provide processing guarantees for Structured Streaming workloads. The checkpoint tracks the information that identifies the query, including state information and processed records.

Checkpoint Component	What It Tracks
Offsets	The source offsets processed in each micro-batch, allowing the query to resume from exactly where it left off without reprocessing data.
Commits	A record of which micro-batches have been committed to the sink, enabling exactly-once semantics.

IMPORTANT WARNING

When you delete the files in a checkpoint directory or change to a new checkpoint location, the next run of the query begins fresh, losing all progress tracking. This is a common production mistake: accidentally pointing two different streaming jobs at the same checkpoint location, or changing a checkpoint path without intending to reset state.

SECTION 05

Asynchronous State Checkpointing

For stateful streaming workloads, such as aggregations or joins with watermarks, checkpointing state can itself become a latency bottleneck.

Characteristic	Synchronous Checkpointing	Asynchronous Checkpointing
Latency	Higher latency for each micro-batch	Reduced latency, since micro-batches can overlap
Restart Behavior	Fast recovery, only the last batch needs to be re-run	Higher restart delay, more than one micro-batch might need to be re-run

Asynchronous state checkpointing maintains exactly-once guarantees but reduces latency by beginning to process the next micro-batch as soon as the previous micro-batch's computation is complete, without waiting for state checkpointing to finish writing. It is available in Databricks Runtime 10.4 LTS and above, but is not compatible with Trigger.Once or Trigger.AvailableNow.

SECTION 06

The Micro-Batch Processing Loop

```
STREAMING SOURCE (files, Kafka, Delta table, etc.)
      |
      v  trigger fires (per configured mode)
READ new data since last recorded offset
      |
      v
PROCESS the micro-batch (transformations, aggregations)
      |
      v
WRITE results to sink
      |
      v
COMMIT offsets + commit log to checkpoint directory
      |
      v  (loop back for next trigger)
```

REMEMBER THIS FOR INTERVIEWS

The checkpoint is what makes this loop resumable and exactly-once: offsets record what's been read, commits record what's been safely written. If a job crashes mid-batch, restarting it replays from the last committed offset rather than either skipping or duplicating data.

SECTION 07

Glossary — Day 21 Terms

Term	Definition
Micro-Batch	A small, discrete chunk of newly arrived data that Structured Streaming

	processes on each trigger.
Trigger	The setting that controls how frequently Structured Streaming checks for and processes new data.
processingTime Trigger	A fixed-interval trigger mode that checks for new data on a defined schedule, such as every 10 seconds.
Trigger.AvailableNow	The current recommended trigger for incremental batch processing, consuming all currently available data across multiple internal batches, then stopping.
Checkpoint	A directory tracking offsets and commits for a streaming query, enabling exactly-once processing and failure recovery.
Asynchronous State Checkpointing	A technique that reduces stateful streaming latency by overlapping micro-batch processing with state checkpoint writes.

SECTION 08

Interview Questions & Answers

Q1. What happens if you don't explicitly set a trigger for a Structured Streaming query?

A. It defaults to processingTime with an interval of 0, which checks for new data every few milliseconds. This can generate a high volume of cloud storage API calls and lead to unexpected cost, so Databricks recommends always configuring an appropriate trigger.

Q2. What is Trigger.AvailableNow, and why has it replaced Trigger.Once?

A. Trigger.AvailableNow consumes all currently available data as an incremental batch, processing it across multiple internal batches for better scalability, with configurable batch sizing. Trigger.Once is deprecated in Databricks Runtime 11.3 LTS and later in favor of it.

Q3. What does a Structured Streaming checkpoint directory actually store?

A. It stores offsets, the source positions processed in each micro-batch, and commits, a record of which micro-batches have been committed to the sink, together enabling the query to resume exactly where it left off and providing exactly-once semantics.

Q4. What happens if you delete or change a streaming query's checkpoint location?

A. The next run of the query begins fresh, with all prior progress tracking lost, meaning previously processed data could be reprocessed depending on the source.

Q5. What is the tradeoff of enabling asynchronous state checkpointing?

A. It reduces per-micro-batch latency by allowing the next micro-batch to start before the previous batch's state checkpoint write completes, but it increases restart delay, since more than one micro-batch might need to be re-run after a failure.

Q6. Is asynchronous state checkpointing compatible with Trigger.AvailableNow?

A. No. Asynchronous progress tracking is not compatible with Trigger.Once or Trigger.AvailableNow; enabling it with those triggers causes the query to fail.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab